CS 106L — Lecture 1

Introduction to C++ · Edited notes from the supplied lecture slides

Learning objectives

- Understand why C++ exists and what trade-offs it makes.
- Recognize the basic structure of a C++ program.
- Understand the roles of a compiler, abstraction, and static typing at a high level.

1. Why C++?

- C++ combines high performance and relatively direct control over computer resources with higher-level language features.
- It is used in areas where performance and control matter, including systems software, browsers, games, networking, and machine-learning infrastructure.
- The language supports abstraction: programmers can organize complex programs using features such as functions, structs, classes, and templates.

A useful summary

> **C++ aims to provide efficiency and expressive abstractions, while giving programmers substantial control—and responsibility.**

2. From machine instructions to high-level languages

Computers execute machine instructions. Assembly language gives humans a readable representation of low-level instructions, but even simple tasks can require many lines of code and the result is often tied to a particular processor or system.

- Assembly: direct control and potential speed, but verbose and difficult to maintain.
- C: a higher-level language whose source code is compiled into machine code; it made systems programming more manageable and portable.
- C++: developed by Bjarne Stroustrup, building on C with additional abstraction mechanisms and higher-level programming features.

The key idea is not to memorize the historical dates, but to understand the trade-off: higher-level constructs make programs easier to express and maintain, while compilers translate them into instructions the computer can execute.

3. A minimal C++ program

```
#include <iostream>

int main() {
    std::cout << "Hello, world!" << std::endl;
    return 0;
}
```

What each part does

- `#include <iostream>` makes stream input/output facilities available.
- `int main()` defines the program's entry-point function.
- `std::cout` writes output to the standard output stream.

- `std::endl` inserts a newline and flushes the stream.
- `return 0;` conventionally indicates successful program completion.

The `std::` prefix names an item in the standard library's namespace. For example, `std::cout` and `std::endl` belong to `std`.

4. Compilation and execution

In a compiled-language workflow, source code is translated by a compiler into a form the machine can execute. The compiler can diagnose many errors before the program runs.

```
source.cpp  --compiler-->  executable  --run-->  output
```

> Compilation does not guarantee that a program is logically correct. It mainly checks whether the code satisfies language and compilation rules; bugs in the program's logic can still remain.

5. Design philosophy of C++

- **Choice and control:** C++ provides mechanisms for low-level control, which also means programmers must use them carefully.
- **Compile-time checking:** types and other language rules allow the compiler to catch many mistakes before execution.
- **Abstraction:** programmers can package details behind interfaces, so users of an abstraction do not need to manage every implementation detail.
- **Multiple paradigms:** C++ supports several ways to structure programs, rather than enforcing a single programming style.

6. Concepts to remember

- C++ is a compiled, statically typed, multi-paradigm language.
- The standard library is accessed through names such as `std::cout`. The namespace prefix is explicit and helps clarify where a name comes from.
- The course's goal is to understand what modern C++ features do and why they exist—not merely to memorize syntax.

Quick self-check

- Why might a programmer choose C++ instead of assembly language?
- What are the roles of `main`, `std::cout`, and `return 0`?
- Why can a program compile successfully and still behave incorrectly?

Editorial note: instructor introductions, lecture schedules, surveys, course administration, promotional examples, and the long assembly-language demonstration have been omitted. Technical ideas and representative examples are retained.